Job Scheduler Overview

To run something on an HPC cluster, like Engaging, you will request resources for your application using a piece of software called the *scheduler*. The scheduler that Engaging uses is Slurm (you'll see and hear "scheduler" and "Slurm" used interchangeably). It is the scheduler's responsibility to allocate the resources that satisfy your request and those of everyone else using the system. This temporary allocation of resources is called a *job*. As with all software, the scheduler uses a specific syntax for requesting resources. This section describes how to work with the scheduler and how to run jobs efficiently.

Partitions

Engaging is a large heterogenous cluster, meaning there are many different types of nodes with different configurations. Some nodes are freely available for anyone at MIT to use, and some have been purchased by labs or departments for priority use by their group. Some nodes are meant for specific types of workloads. Nodes are grouped together in *partitions*, which designate who can access them and what they should be used for. Different partitions may have different sets of rules about how many resources you can use and how long your jobs can run on them.

The standard partitions that the full MIT community has access to are:

Partition Name	Purpose	Hardware Type(s)	Max Time Limit	Base Resource Limit
`mit_normal`	Longer running batch and interactive jobs that do not need a GPU	CPU only	12 hours	96 cores
`mit_normal_gpu`	Batch and interactive jobs that need a GPU	GPUs (L40S, H100, H200)	6 hours	2 GPUs, 32 cores

Partition Name	Purpose	Hardware Type(s)	Max Time Limit	Base Resource Limit
`mit_quicktest`	Short batch and interactive jobs, meant for testing	CPU only	15 minutes	48 cores
`mit_preemptable`	Low-priority preemtable jobs- jobs that may be stopped by another job with higher priority	CPU-only, GPUs (A100, L40S, H100, H200)	48 hours	1024 cores, 4 GPUs

To see a summary of the nodes in each of these partitions, take a look at our Available Resources page.

Older Partitions

There are a few additional partitions that contain older nodes. These nodes run on a different operating system (Centos 7) than the ones above and therefore have a different software stack. Software built or installed on Rocky 8 or newer nodes will most likely not work on these older nodes. These partitions include `sched_mit_hill`, `newnodes`, `sched_any`, `sched_engaging_default`, and `sched_quicktest`.

If you are part of a group that has purchased nodes you may see additional partitions. They will be named based on your PI's Kerberos or your group's name, depending on who purchased the nodes.

To see which partitions you have access to, run the `sinfo` command:

```
sinfo
PARTITION    AVAIL  TIMELIMIT  NODES  STATE NODELIST
mit_normal      up   12:00:00      2   resv node[2704-2705]
mit_normal      up   12:00:00     30   idle node[1600-1625,1706-1707,1806-1807]
mit_normal_gpu  up   12:00:00      1    mix node2906
mit_normal_gpu  up   12:00:00      5   idle node[1706-1707,1806-1807,2804]
mit_quicktest   up      15:00     26   idle node[1600-1625]
```

```
mit_preemptable up 2-00:00:00      1    mix node2906
mit_preemptable up 2-00:00:00     27   idle node[1600-1625,2804]
```

The `sinfo` command will tell you the names of the partitions you have access, what their time limits are, how many nodes are in each state, and the names of the nodes in the partitions.

Preemptable Jobs

We provide the `mit_preemptable` partition so that nodes owned by a group or PI can be used by researchers outside that group when those nodes are idle. When someone from the group that owns the node runs a job on their partition, the scheduler will stop, or preempt, any job that is running on the lower-priority `mit_preemptable` partition. Jobs running on `mit_preemptable` should be checkpointed so that they don't lose their progress when the job is stopped.

To make your batch jobs requeue if they are stopped on `mit_preemptable`, add the `--requeue` flag to your job submission. If your job has a recent checkpoint it should pick up where it left off once additional resources are available. You can read more about job flags below.

Checking Available Resources

To see what resources are available run `sinfo`. The `sinfo` command will show how many nodes are in each state. Nodes in "idle" state have all cores available, nodes in "mix" state have some cores available, and nodes in "alloc" state have no cores or other resources available.

```
sinfo -p mit_normal
PARTITION  AVAIL  TIMELIMIT  NODES  STATE  NODELIST
mit_normal    up   12:00:00      2   resv  node[2704-2705]
mit_normal    up   12:00:00      1   mix   node1707
mit_normal    up   12:00:00      1   alloc node1708
mit_normal    up   12:00:00     29   idle  node[1600-1625,1706,1806-1807]
```

Common node states are:

- `idle`: nodes that are fully available
- `mix`: nodes that have some, but not all, resources allocated
- `alloc`: nodes that are fully allocated
- `resv`: nodes that are reserved and only available to people in their reservation
- `drained`: nodes that are unavailable for use per system administrator request, usually for maintenance purposes. It is shown as `drain*` due to the default 5-column limit.

- `drng` : nodes that are currently allocated a job, but will not be allocated additional jobs. The node state will be changed to state drained when the last job on it completes.
- `down` : nodes that are unavailable for use.
- `plnd` : nodes that are planned by the backfill scheduler for a higher priority job

You can also use `sinfo` to see what resources each node has. The output can be quite long so we recommend limiting to a specific partition(s). For example, to see what node types are in the `mit_normal` and `mit_normal_gpu` partitions, run the command:

```
sinfo -p mit_normal,mit_normal_gpu -O Partition,Nodes,CPUs,Memory,Gres -e
```

In the output you'll see a summary of how many nodes of each configuration is in the partition. You can include multiple partitions by providing a comma separated list to the `-p` flag. The output shows the partition, number of nodes, number of CPUs, amount of Memory (in MB), and any GPUs available on the node:

```
PARTITION           NODES               CPUS                MEMORY
GRES
mit_normal          32                  96                  386000
(null)
mit_normal          2                   64                  386000
(null)
mit_normal          6                   192                 1547000
(null)
mit_normal          12                  96                  386223
(null)
mit_normal_gpu      1                   64                  1031314
gpu:h100:4(S:0-1)
mit_normal_gpu      8                   120                 2063213
gpu:h200:8(S:0-1)
mit_normal_gpu      49                  64                  1031314
gpu:l40s:4(S:0-1)
```

Running Jobs

How you run your job depends on the type of job you would like to run. There are two "modes" for running jobs: interactive and batch jobs. Interactive jobs allow you to run interactively on a compute node in a shell. Batch jobs, on the other hand, are for running a pre-written script or executable. Interactive jobs are mainly used for testing, debugging, and interactive data analysis. Batch jobs are the traditional jobs you see on an HPC system and should be used when you want to run a script that doesn't require that you interact with it.

Job Flags

When you start any type of job you specify what resources you need for your job, including cores, memory, GPUs, and other features. You also specify which partition you would like your job to run on. If you don't specify any of these you will get the default resources: 1 core, a small amount of memory, no GPUs, and it will run on the current default partition. See the page on Requesting Resources for the flags to use to request different types of resources.

Interactive Jobs

The basic command for requesting an interactive job on the `mit_normal` partition is:

```
salloc -p mit_normal
```

The `-p mit_normal` is a flag that is passed to the scheduler, `-p` specifies the partition. This command will allocate 1 core on a node in the `mit_normal` partition. For example:

```
[user01@orcd-login001 ~]$ salloc -p mit_normal
salloc: Pending job allocation 60159437
salloc: job 60159437 queued and waiting for resources
salloc: job 60159437 has been allocated resources
salloc: Granted job allocation 60159437
salloc: Waiting for resource configuration
salloc: Nodes node1806 are ready for job
[user01@node1806 ~]$
```

Notice how the command prompt changes from `[user01@orcd-login001 ~]$` to `[user01@node1806 ~]$`. This indicates that `user01` has started an interactive job on `node1806` and any commands issued will run on this node.

Batch Jobs

Batch jobs are used to run pre-written scripts or run commands that do not need input from you throughout the run. The first step to running a batch job is to write a job script. Job scripts can be "launched" with the `sbatch` command:

```
sbatch myscript.sh
```

When you run this command the scheduler will look for the resources requested in the script, allocate those resources to your job, run your script on those resources, and then release those resources once your script completes or the time limit is reached.

Here is an example job script:

```
#!/bin/bash

# Job Flags
#SBATCH -p mit_normal

# Set up environment
module load miniforge

# Run your application
python myscript.py
```

This script requests the same resources as the interactive job above: 1 cpu core on the `mit_normal` partition. The `#SBATCH -p mit_normal` may look like a comment but it is not, it is a directive to the scheduler to run with the specified flags. Note that this is the same flag used in the interactive job example above. It then sets up the job environment to use python with the `miniforge` module, and then runs a python script. In general, the same steps and commands you would use to run your job in an interactive job you can put in your job script.

You can think of job scripts as having three sections:

1. Scheduler/Job flags: This is where you request your resources using Slurm flags.
2. Set up your environment: Load any modules you need, set environment variables, etc. It is better to set this in your job scripts to ensure consistent environments across jobs. We don't recommend putting these commands in your `.bashrc` or running them at the command line before you launch your job.
3. Run your code or application as you would from the command line.

Checking Job Status

To see all your currently running and pending jobs run the `squeue --me` command:

```
squeue --me
             JOBID PARTITION     NAME     USER ST       TIME  NODES
NODELIST(REASON)
          60735877 mit_norma interact username  R      29:32      1 node1707
```

each line in the output is a different job. The default fields are:

- `JOBID`: The Job ID, each job has a unique ID that can be used to identify it.
- `PARTITION`: The partition the job is running in

- `NAME`: The name of the job. By default this is the name of your job script, or "interactive" for interactive jobs.
- `USER`: Your username
- `ST`: The job status. Common statuses are `R` for "Running", `PD` for "Pending", and `CG` for completing. Jobs in pending state have the reason the job is pending listed in the final `NODELIST(REASON)` column.
- `TIME`: The amount of time the job has been running for.
- `NODES`: The number of nodes your job is using.
- `NODELIST(REASON)`: If your job is running this is the name of the node or nodes that your job is running on. If your job is pending this lists the reason the job is pending. Common pending reasons are "Resources", meaning the resources you've requested aren't yet available, "Priority", meaning there is someone else ahead of you in line to use the resources you've requested. Other reasons are listed in the Slurm documentation.

Stopping Jobs

You can stop running or pending jobs with the `scancel` command. You can stop an individual job by providing a job id:

```
scancel 123456
```

or a list of jobs by separating job IDs with a comma:

```
scancel 123456,123457
```

You can also stop all of your jobs by providing your username:

```
scancel -u USERNAME
```

where `USERNAME` is your username.

Retrieving Job Stats

You can get information above currently and previously run jobs with the `sacct` command. This command can be very helpful for troubleshooting job issues, and is particularly helpful to check that you are getting the resources you expect allocated to your job.

Running `sacct` with no flags will show some basic information about the jobs that you've run in the past day:

```
[USERNAME@orcd-login001 ~]$ sacct
JobID           JobName  Partition    Account  AllocCPUS      State ExitCode
------------ ---------- ---------- ---------- ---------- ---------- --------
60764362     interacti+ sched_any+ mit_gener+          1 CANCELLED+      0:0
60764363     interacti+ mit_normal mit_gener+          1  COMPLETED      0:0
60764363.in+ interacti+            mit_gener+          1  COMPLETED      0:0
60764363.ex+     extern            mit_gener+          1  COMPLETED      0:0
60764366     interacti+ mit_normal mit_gener+          1    RUNNING      0:0
60764366.in+ interacti+            mit_gener+          1    RUNNING      0:0
60764366.ex+     extern            mit_gener+          1    RUNNING      0:0
```

You can select which fields are shown with the `-o` (output) flag, for example:

```
[USERNAME@orcd-login001 ~]$ sacct -o
JobID,JobName,AllocCPUs,NodeList,Elapsed,State
JobID           JobName  AllocCPUS        NodeList    Elapsed      State
------------ ---------- ---------- --------------- ---------- ----------
60764362     interacti+          1   None assigned   00:00:00 CANCELLED+
60764363     interacti+          1        node1806   00:00:19  COMPLETED
60764363.in+ interacti+          1        node1806   00:00:19  COMPLETED
60764363.ex+     extern          1        node1806   00:00:19  COMPLETED
60764366     interacti+          1        node1806   00:01:40  COMPLETED
60764366.in+ interacti+          1        node1806   00:01:39  COMPLETED
60764366.ex+     extern          1        node1806   00:01:40  COMPLETED
```

There are many fields that give a lot of information about your jobs. Running the `sacct -e` command will show all of them. A few that can be helpful are:

- `AllocCPUs`, `AllocNodes` - Number of CPUs, number of nodes, or GPUs allocated to your job.
- `ReqTRES` - Requested "trackable resources" (TRES). This is a fairly long string that will include any GPUs requested for the job. If the string is too long to display add `%60` to show 60 characters (`ReqTRES%60`), you can adjust the number of characters as needed.
- `NodeList` - The list of nodes that your job ran on. This is helpful to see whether your jobs are consistently failing on the same node. If so, reach out to orcd-help-engaging@mit.edu and let us know which node seems to be having an issue.
- `Start`, `End`, `Elapsed` - The start time, end time, and total amount of time your job ran for.
- `State`, `ExitCode` - The Job State and Exit Code. If the job failed there may be an exit code that can help determine why the job failed.
- `MaxRSS` - The peak memory utilization of your job. This number can be used to fine-tune how much memory to request for your job. See the section on requesting memory for more

information.

You can specify a particular job with the `-j` flag (`sacct -j 60764366` for example). You can also specify a start and end time if you are interested in seeing jobs that ran more than a day ago. Use the `--starttime` and `--endtime` flags to specify a date range, using the format `MMDDYY`, for example `sacct --starttime=010125 --endtime=010225` retrieves all your jobs that ran on the first two days of 2025.

See the Slurm sacct documentation for more information on flags and fields.

January 16, 2026